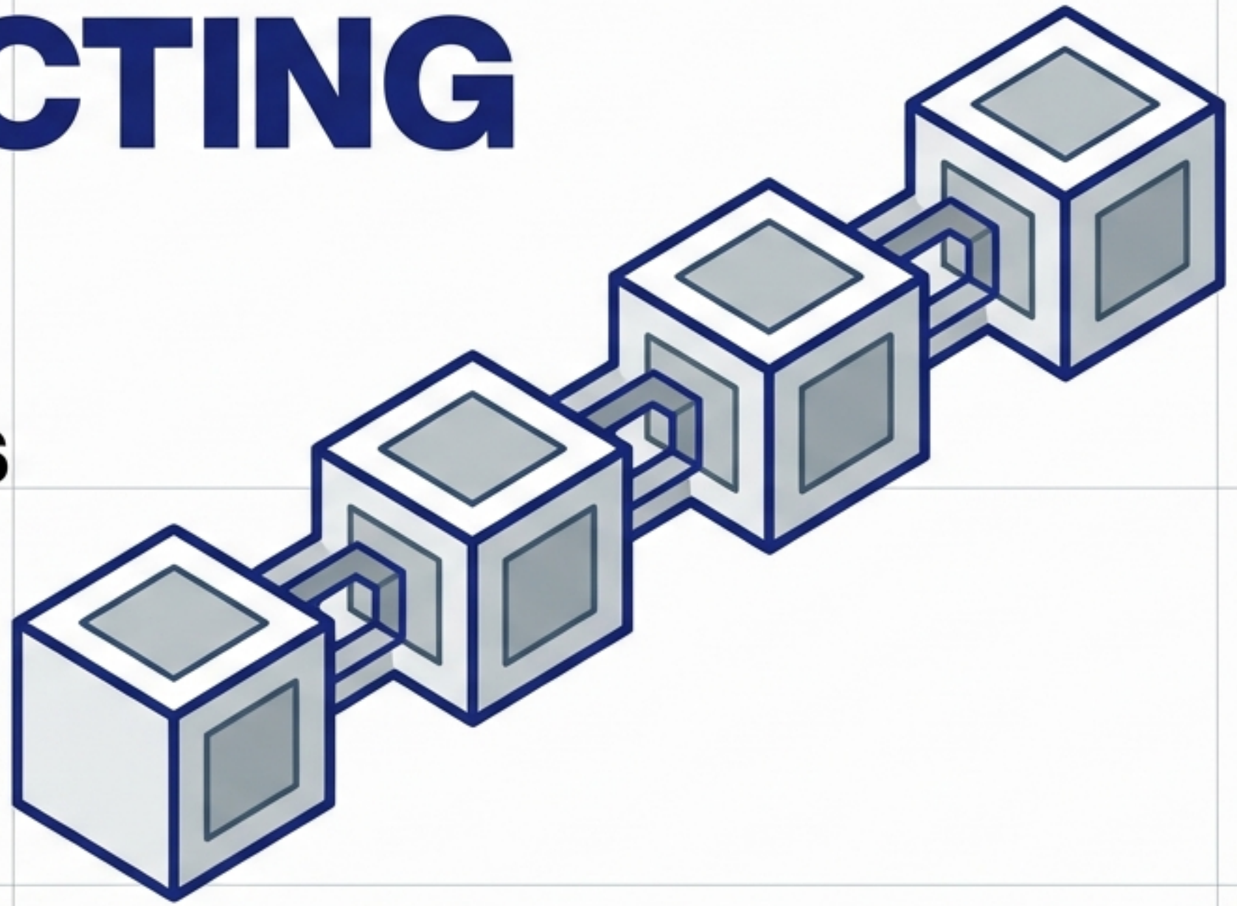
DECONSTRUCTING LANGCHAIN

The 6 Core Components

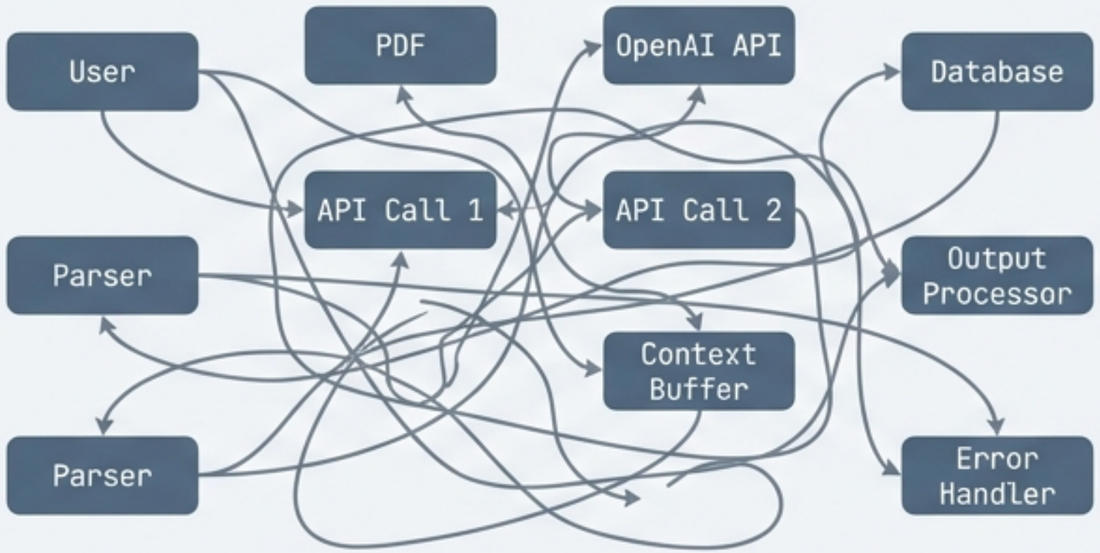
A conceptual roadmap for building robust GenAI applications.



Context: An engineering framework guide for developers transitioning from prompt engineering to application architecture.

Why LangChain? The Need for Orchestration

The Pain: Raw LLM Integration



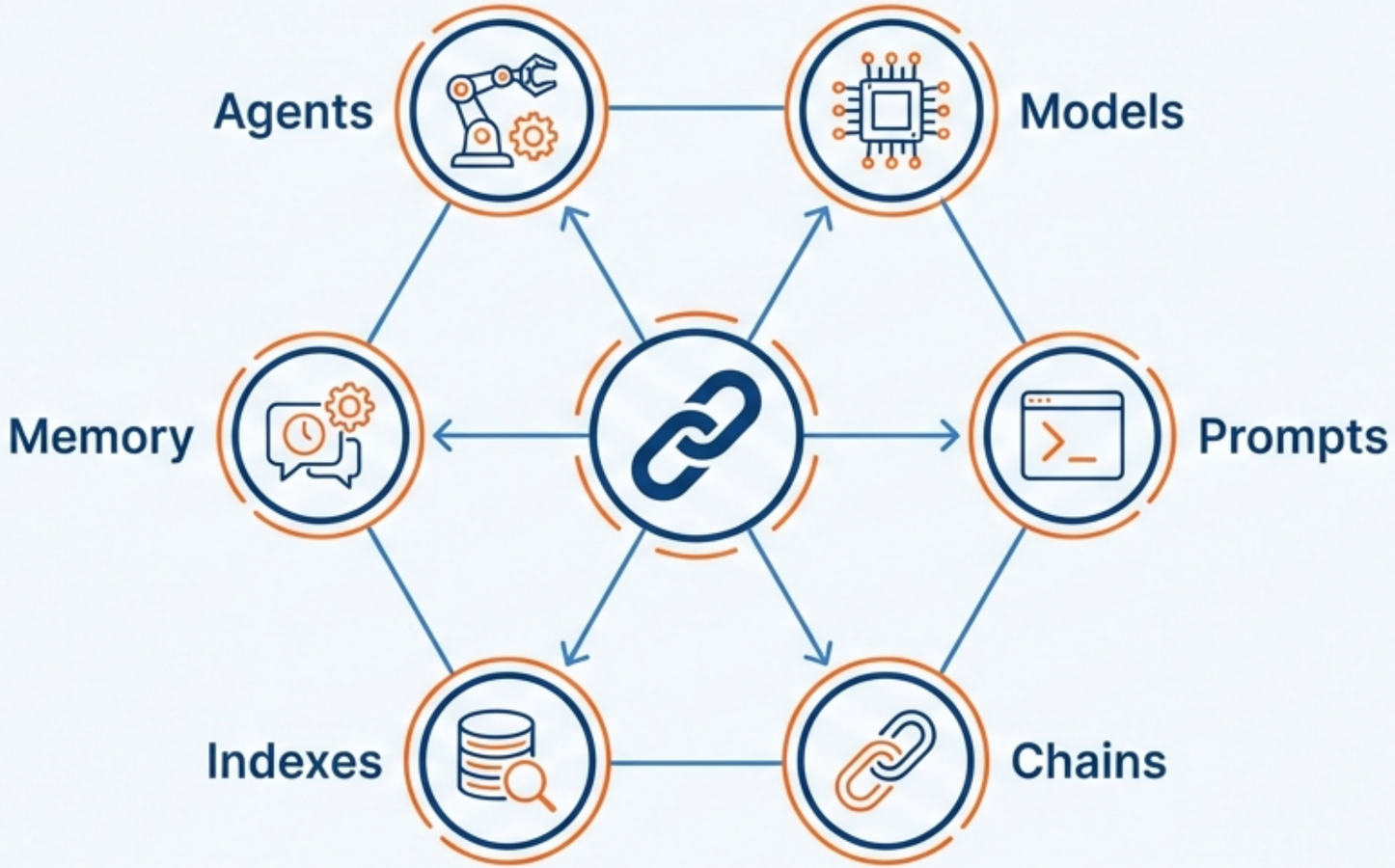
Building apps from scratch requires **managing complex API calls, manual data parsing**, and **context management.** It leads to fragile, unmaintainable code.

The Solution: LangChain Orchestration

An open-source framework that manages the pipeline efficiently. The output of one component automatically becomes the input of the next.

Minimal Code, Maximum Output.

The Blueprint: Six Pillars of the Framework

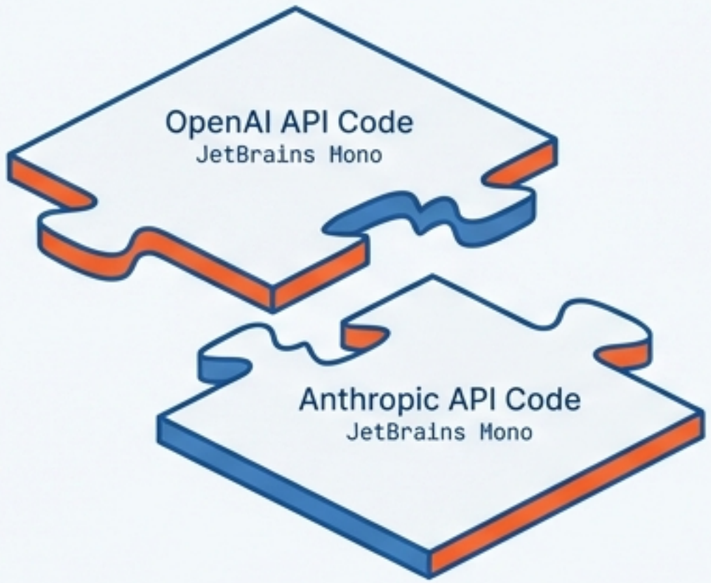


Mastery of these six components covers the majority of the framework's functionality, enabling the transition from simple scripts to **complex, autonomous applications**.

1. Models | The Universal Interface

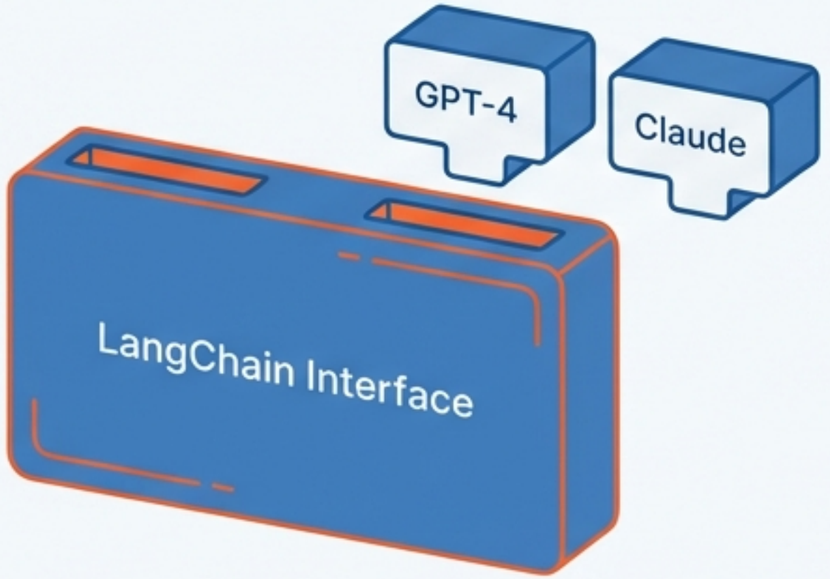
The Problem: API Inconsistency vs. The Solution: Standardisation

The Problem: API Inconsistency



Vendor Lock-in. Changing providers requires rewriting the codebase.

The Solution: Standardisation



Standardised Interaction. Switch from GPT-4 to Claude with a single line of code.

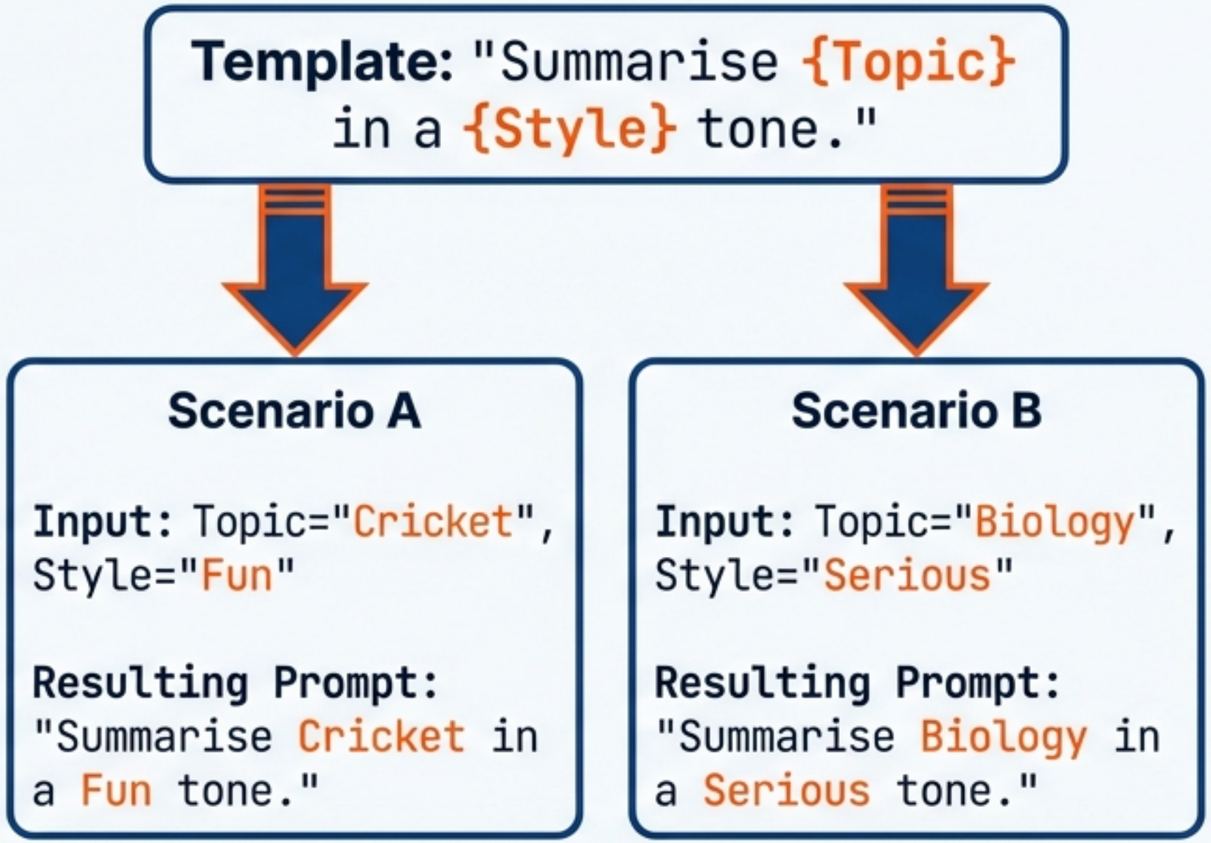
```
from langchain.chat_models import ChatOpenAI
# vs
from langchain.chat_models import ChatAnthropic

model = ChatOpenAI() // Change this one line to switch providers
result = model.predict('Hello World')
```

Distinction: Language Models (Text-in/Text-out) vs. Embedding Models (Text-in/Vector-out).

2. Prompts | Engineering the Input

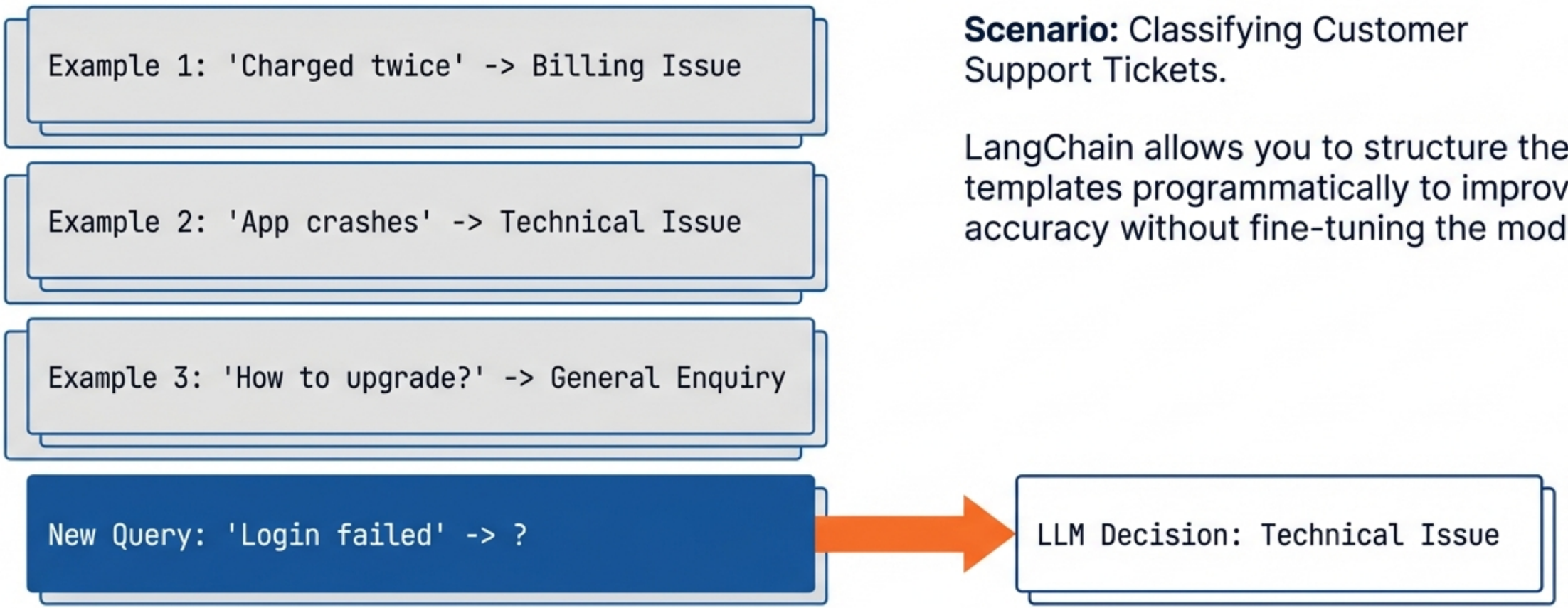
Hardcoding strings is not scalable. LLMs are sensitive; slight wording changes alter outputs. The solution is Dynamic, Reusable Templates.



Role-Based Prompting: Guide specific outputs by assigning personas (e.g., 'You are an experienced Doctor...').

3. Advanced Prompting | The ‘Few-Shot’ Technique

Teaching the LLM by example before asking the question.

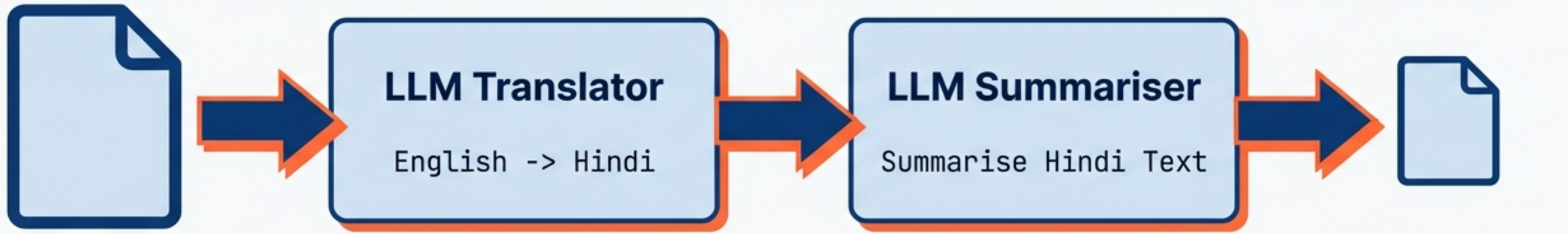


Scenario: Classifying Customer Support Tickets.

LangChain allows you to structure these templates programmatically to improve accuracy without fine-tuning the model.

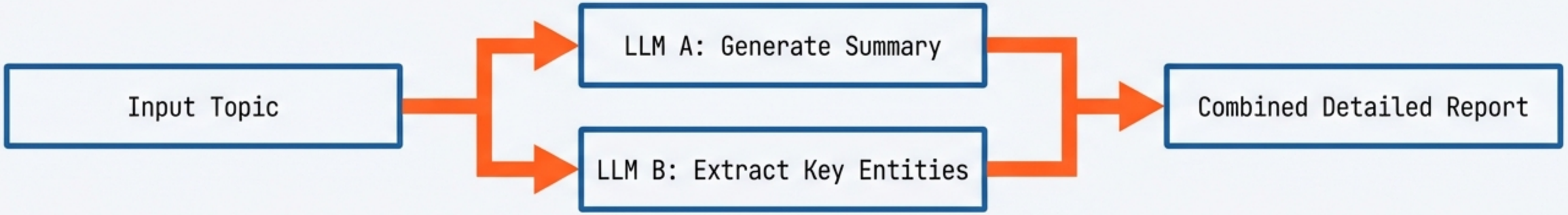
3. Chains | Building the Pipeline

Passing data manually between steps is tedious. Chains act as wrappers that link steps together: The output of one component automatically becomes the input of the next.



Beyond Linear: Complex Chain Architectures

Parallel Chains



Conditional Chains

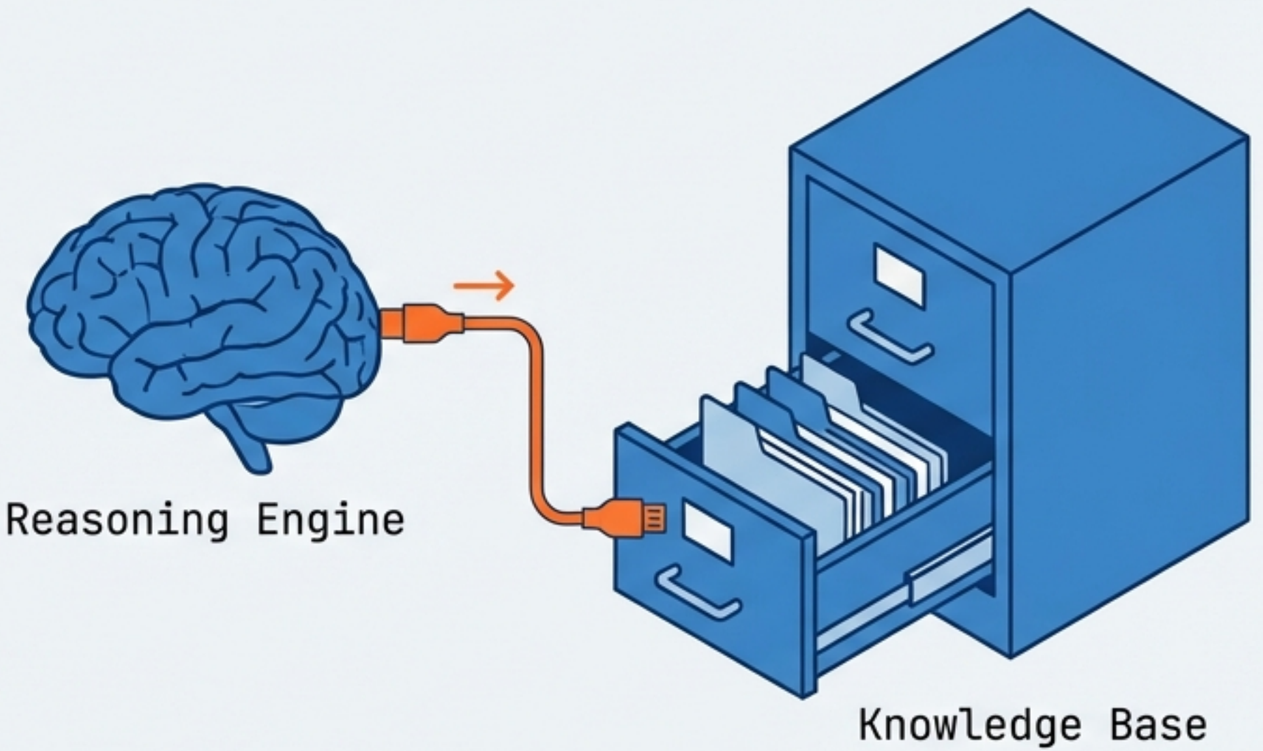
Chains allow for logic-based routing, enabling true application complexity beyond simple sequential processing.

4. Indexes | Connecting to External Knowledge

The Limitation: LLMs are trained on public internet data. They do not know private data (e.g., “What is the company leave policy?”). This leads to hallucinations.

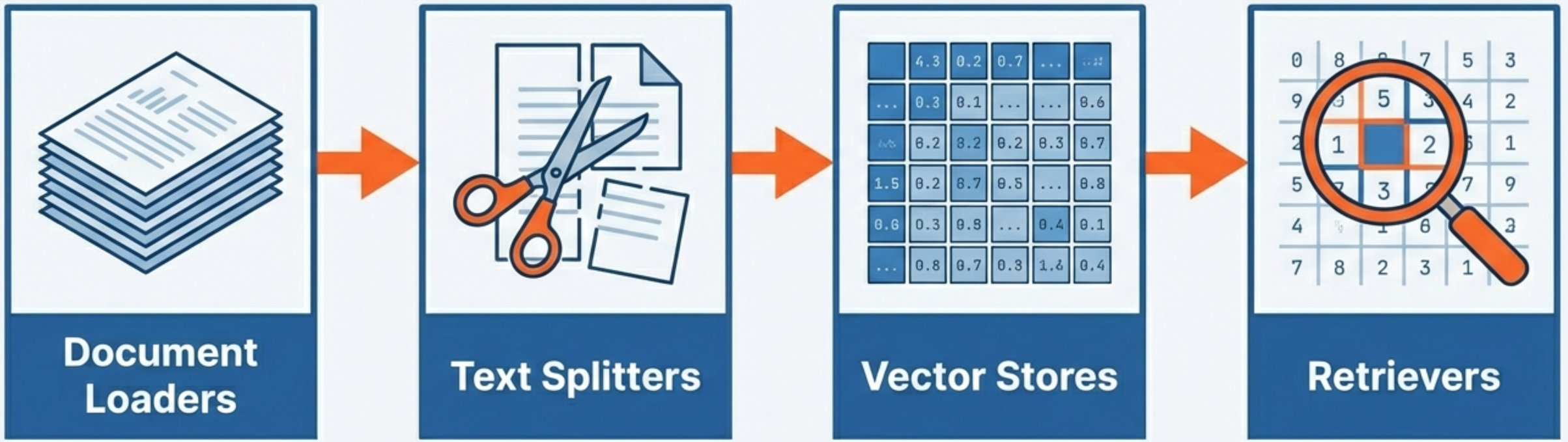
The Solution: RAG (Retrieval-Augmented Generation). Connecting LLMs to external sources like PDFs, Databases, and Websites.

The Open Book Exam



The LLM doesn’t memorise the book; it learns how to look up the relevant page when asked a question.

The Architecture of Retrieval (RAG)



Ingest raw data (PDFs/Web)

Break into chunks

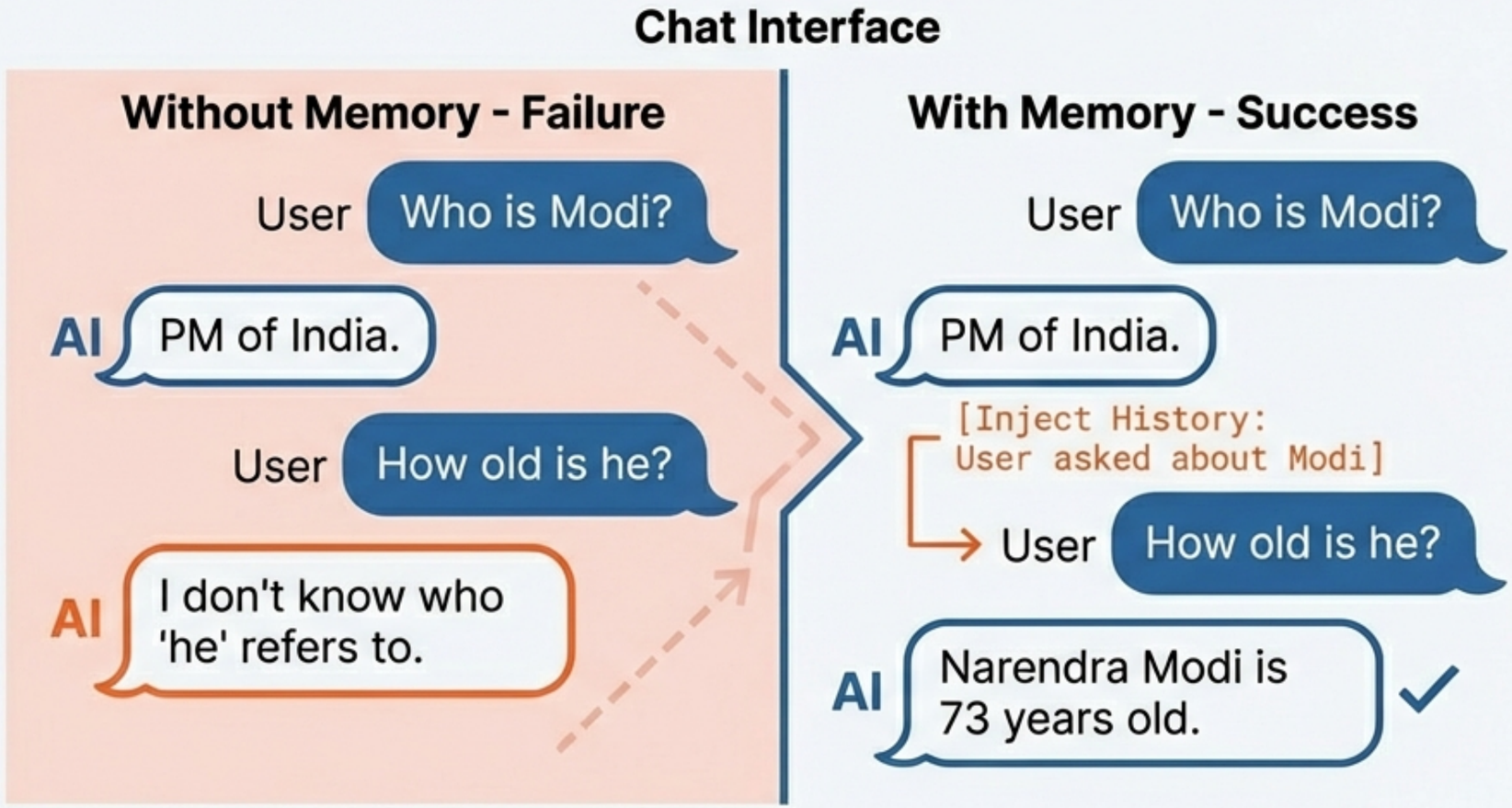
Store embeddings for semantic search

Fetch relevant chunks for the LLM

This architecture enables context-aware answers based on private data.

5. Memory | Solving Amnesia

LLM APIs are **Stateless**. They treat every request as an independent event.



1. Conversation Buffer
(Stores everything)

2. Window Memory
(Stores last N messages)

3. Summary Memory
(Stores the gist)

6. Agents | From Passive Talk to Active Action

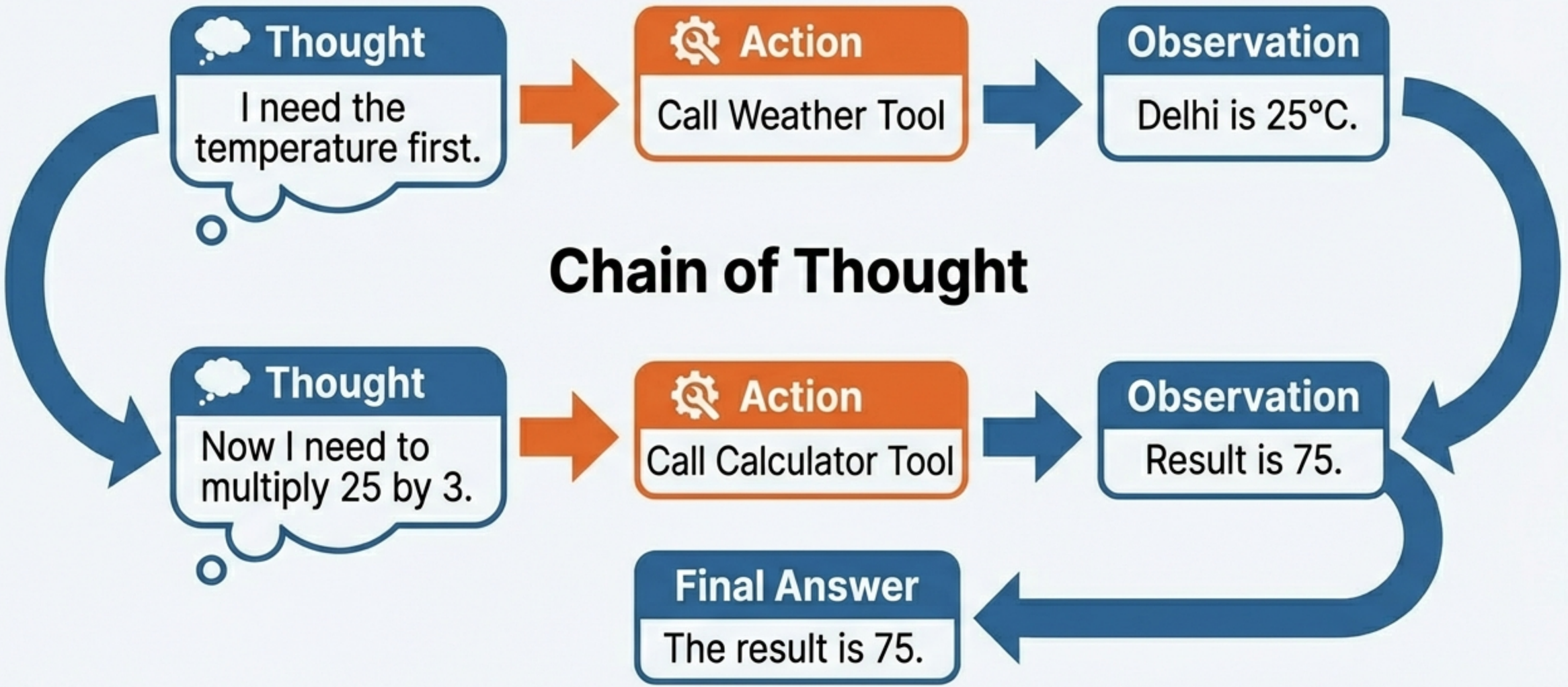
REASONING + TOOLS = AGENCY

 **ChatBots** Passive	 **Agents** Active
Suggests a holiday destination based on training data. Example: “You should visit Shimla.”	Checks real-time prices and performs actions. Example: “I found a flight for ₹5000. Shall I book it?”

The **Agent** has access to a toolkit (Search, Calculator, API) and autonomously decides which tool to use.

Agents in Action: The Reasoning Loop

User Query: “Multiply the current temperature of Delhi by 3.”



Your GenAI Toolkit: A Summary

MODELS The Interface (Switch providers easily)	**PROMPTS** The Instructions (Dynamic templates)	**CHAINS** The Pipeline (Sequence of events)
INDEXES The Knowledge (Access private data)	**MEMORY** The Context (Remembering history)	**AGENTS** The Actor (Reasoning + Tools)

These components are the building blocks. The next step is writing the code to connect them.